

Reconhecimento de Objetos Usando YOLO

Projeto 1

MS960/MT862: Tópicos em Aprendizado de Máquina (Deep Learning)

Integrante 1 (RA: ...)

Integrante 2 (RA: ...)

Integrante 3 (RA: ...)

Integrante 4 (RA: ...)

Prof. João Florindo

IMECC/UNICAMP

6 de maio de 2026

Resumo

Este relatório descreve a implementação e a análise do algoritmo YOLOv3 para detecção de objetos em imagens, desenvolvida em Python com o framework PyTorch. As funções de Intersecção sobre União (IoU) e de Supressão Não Maximal (NMS) foram construídas manualmente, seguindo as orientações do enunciado e a errata fornecida pelo professor. Discutimos a arquitetura DarkNet-53 com três cabeças de detecção, apresentamos os resultados obtidos sobre a base de imagens fornecida e analisamos criticamente o impacto dos limiares de NMS e IoU no número e na qualidade das detecções.

1 Introdução

A detecção de objetos é uma das tarefas fundamentais da visão computacional. Diferentemente da classificação, que apenas atribui um rótulo a uma imagem inteira, a detecção exige duas respostas simultâneas: identificar *o que* está presente e *onde* está, em geral por meio de uma caixa delimitadora (*bounding box*) e de um nível de confiança associado a uma classe. Essa combinação de tarefas faz da detecção um problema mais difícil e, ao mesmo tempo, com aplicações práticas mais ricas [8, 14].

A relevância da detecção de objetos cresceu de forma marcante na última década, impulsionada pelo avanço das redes neurais profundas e pela disponibilidade de bases anotadas de larga escala. Dentre as aplicações mais comuns estão veículos autônomos, em que a localização precisa de pedestres, ciclistas e outros automóveis é crítica para a tomada de decisão; sistemas de vigilância, que dependem de rastreamento robusto em tempo real; varejo automatizado, em que prateleiras e produtos precisam ser identificados sem intervenção humana; e diagnóstico assistido em imagens médicas, no qual lesões e estruturas anatômicas são localizadas para apoiar o trabalho clínico.

Em todos esses cenários, requisitos de latência e precisão coexistem e, com frequência, são conflitantes.

Os métodos modernos de detecção podem ser organizados em três grandes famílias. A primeira é a dos detectores em *dois estágios*, representada pelas variantes da R-CNN [9, 10]: primeiro propõem-se regiões candidatas e em seguida cada região é classificada e refinada. Esses métodos costumam atingir alta precisão, ao custo de uma latência maior. A segunda família é a dos detectores em *estágio único*, na qual se enquadram o SSD [11] e a família YOLO [1, 3], que tratam a detecção como um único problema de regressão sobre uma grade de células e oferecem boa precisão com latência muito baixa. A terceira família, mais recente, é a dos detectores baseados em Transformers, com destaque para o DETR [12], que reformula a detecção como um problema de atribuição entre conjuntos sem necessidade de NMS explícito.

Dentro deste contexto, o YOLO se destaca pela proposta original de Redmon et al. [1]: realizar uma única passagem pela imagem (*You Only Look Once*) e produzir, em paralelo, todas as caixas e classes. Essa estratégia permite operação em tempo real e, ao mesmo tempo, mantém precisão competitiva. Neste projeto adotamos a versão YOLOv3 [3], por ser a versão correspondente aos pesos pré-treinados disponibilizados no enunciado e por já incorporar dois ingredientes que se tornaram padrão no campo: detecção em múltiplas escalas via *Feature Pyramid Network* (FPN) [6] e backbone profundo com conexões residuais [7].

2 Algoritmo YOLO

2.1 Visão geral e intuição

O YOLO formula a detecção como um problema de regressão. Em vez de propor regiões e depois classificá-las, ele divide a imagem em uma grade $S \times S$ e atribui a cada célula a responsabilidade pelos objetos cujo centro caia dentro dela. A cada célula são associados B *anchor boxes*, ou seja, caixas de referência com largura e altura fixadas previamente. A rede aprende a corrigir essas caixas de referência por meio de pequenos deslocamentos, em vez de prever as coordenadas absolutas do zero [2]. Essa decisão simplifica a otimização e estabiliza o treinamento.

A saída da rede para cada *anchor* de cada célula contém:

- dois deslocamentos (t_x, t_y) que, depois de uma sigmoide, posicionam o centro da caixa dentro da célula;
- dois fatores (t_w, t_h) que escalam exponencialmente as dimensões da *anchor*;
- uma confiança t_o , conhecida como *objectness*, que indica se há de fato um objeto naquela posição;
- um vetor de probabilidades por classe.

Formalmente, dados a célula (c_x, c_y) e a *anchor* de tamanho (p_w, p_h) , a caixa final é dada por

$$b_x = \sigma(t_x) + c_x, \quad (1)$$

$$b_y = \sigma(t_y) + c_y, \quad (2)$$

$$b_w = p_w \cdot e^{t_w}, \quad (3)$$

$$b_h = p_h \cdot e^{t_h}, \quad (4)$$

em coordenadas da grade. A sigmoide em t_x e t_y garante que o centro permaneça dentro da célula, e a exponencial em t_w e t_h assegura dimensões positivas.

2.2 Arquitetura do YOLOv3

A versão YOLOv3 é composta por dois grandes blocos. O primeiro é o *backbone*, chamado DarkNet-53, formado por uma sequência de convoluções 3×3 e 1×1 intercaladas por blocos residuais. Esse backbone tem 53 camadas convolucionais e foi inspirado tanto na DarkNet-19 do YOLOv2 quanto na ResNet, combinando profundidade com a estabilidade dos atalhos residuais. O segundo bloco é a *cabeça* de detecção, que utiliza uma estrutura de pirâmide de características (FPN) para emitir predições em três escalas diferentes:

- escala 13×13 : resolução mais grosseira, dedicada a objetos grandes;
- escala 26×26 : resolução intermediária;
- escala 52×52 : resolução fina, dedicada a objetos pequenos.

Cada escala recebe três *anchors* dimensionadas por *k-means* sobre o conjunto COCO [8], totalizando nove âncoras. Os mapas de saída têm $3 \times (5 + C)$ canais, onde $C = 80$ é o número de classes do COCO; o fator 3 corresponde às três *anchors* por célula e o 5 corresponde a $(t_x, t_y, t_w, t_h, t_o)$.

A rede de detecção total tem cerca de 62 milhões de parâmetros. Em nossa implementação, o resumo automático produzido pelo notebook conta 75 camadas `Conv2d`, 72 camadas `BatchNorm2d` e 23 blocos residuais, valores compatíveis com o que é descrito na literatura.

2.3 Função de perda e treinamento

Embora neste projeto utilizemos os pesos já treinados sobre o COCO, vale resumir a função de perda do YOLOv3 para fins de discussão. Ela é uma soma ponderada de três termos: regressão das coordenadas das caixas (com erro quadrático sobre t_x, t_y, t_w, t_h), entropia binária para a *objectness* e entropia binária independente por classe (em vez de softmax, o que permite atribuir múltiplos rótulos quando a hierarquia de classes assim o permite) [3]. Apenas as âncoras com maior IoU em relação a cada objeto rotulado contribuem para a regressão das coordenadas, evitando ambiguidades.

2.4 Pós-processamento: IoU e NMS

Após a passagem direta, cada uma das três escalas produz um conjunto denso de caixas candidatas. Antes de exibir os resultados, dois passos de pós-processamento são essenciais: filtragem por confiança e Supressão Não Maximal (NMS).

A IoU entre duas caixas A e B é definida como

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad (5)$$

e mede o grau de sobreposição entre regiões. Em nossa implementação manual, a IoU é calculada de forma vetorizada entre uma caixa de referência e um conjunto de caixas, determinando os cantos da intersecção, calculando a área correspondente e dividindo pela área da união.

O NMS, por sua vez, evita que múltiplas caixas redundantes sejam exibidas para um mesmo objeto. Conforme observado pelo professor na errata enviada à turma, a operação deve ser realizada de forma independente por classe, pois objetos de classes diferentes podem legitimamente se sobrepor (um cachorro montado em um cavalo, por exemplo). O algoritmo, em pseudocódigo, é o seguinte:

1. Para cada classe presente entre as caixas filtradas:
 - (a) Ordene as caixas dessa classe por confiança decrescente.
 - (b) Tome a caixa de maior confiança e mantenha-a.
 - (c) Calcule a IoU dessa caixa contra todas as demais da mesma classe.
 - (d) Descarte aquelas com IoU acima de um limiar τ (caixas redundantes).
 - (e) Repita o processo com as caixas restantes até esvaziar a lista.
2. Junte os índices mantidos por todas as classes.

A escolha de τ é um *trade-off*: valores baixos eliminam caixas legítimas que estejam próximas (objetos vizinhos), enquanto valores altos preservam duplicatas. Da mesma forma, o limiar de confiança $s_{\min}$ aplicado antes do NMS controla quão permissivo é o detector: valores altos descartam predições inseguras, mas podem perder objetos verdadeiros; valores baixos aumentam a cobertura ao custo de mais falsos positivos.

3 Implementação

A implementação completa está distribuída em dois arquivos principais: o notebook `YOLO_Projeto1.ipynb`, que contém a definição da arquitetura, o pipeline de detecção e os experimentos; e o arquivo `utils.py`, que isola as funções manuais de IoU e NMS, conforme solicitado.

3.1 Pipeline de inferência

A entrada do modelo tem tamanho fixo 416×416 . Para preservar a razão de aspecto da imagem original (ponto importante para evitar distorções nas caixas previstas), aplicamos uma técnica chamada *letterbox*: a imagem é reescalada de modo que o lado maior ocupe os 416 pixels e o lado menor é completado com cinza neutro. Após a passagem pela rede, as coordenadas previstas em $[0, 1]$ são revertidas para o sistema da imagem original por meio da função `reverter_escala_caixas`.

A decodificação das saídas (`decode_yolo`) converte os tensores brutos em caixas $[y_1, x_1, y_2, x_2]$, scores e índices de classe. Em seguida, aplicamos um filtro por *score* mínimo e a função `manual_nms` para obter as caixas finais.

3.2 Funções manuais de IoU e NMS

A Figura 1 mostra um trecho de `utils.py`, com a chamada de `manual_nms` já adaptada para receber o vetor de classes, conforme a errata.

```
1 def manual_nms(boxes, scores, classes, iou_threshold=0.45):
2     keep_global = []
3     classes_unicas = torch.unique(classes)
4
5     for cls in classes_unicas:
6         mask_cls = (classes == cls)
7         idx_cls = torch.nonzero(mask_cls, as_tuple=False).flatten()
8         boxes_cls = boxes[mask_cls]
9         scores_cls = scores[mask_cls]
10
11         ordem = torch.argsort(scores_cls, descending=True)
12         keep_local = []
13         while ordem.numel() > 0:
14             i = ordem[0].item()
15             keep_local.append(idx_cls[i].item())
16             if ordem.numel() == 1:
17                 break
18             ious = manual_iou(boxes_cls[i], boxes_cls[ordem[1:]])
19             mantidas = torch.nonzero(ious <= iou_threshold,
20                                     as_tuple=False).flatten()
21             ordem = ordem[1:][mantidas]
22         keep_global.extend(keep_local)
23     # ... reordenacao final por score ...
```

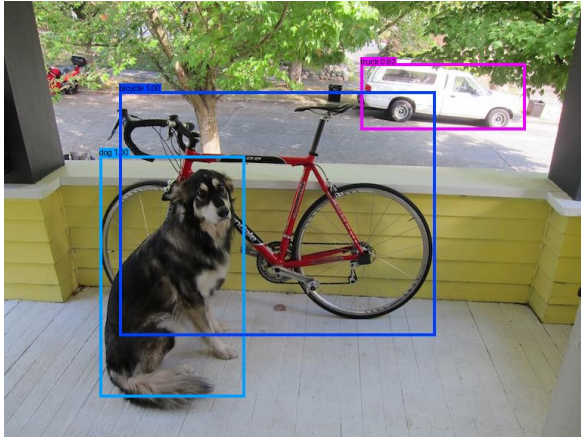
Figura 1: Trecho de `utils.py`: NMS manual com processamento independente por classe.

4 Resultados

Aplicamos o detector com os limiares padrão $s_{\min} = 0,5$ e $\tau_{\text{IoU}} = 0,45$ a todas as imagens da base. A Tabela 1 resume o número de objetos detectados por imagem e os respectivos rótulos. A Figura 2 apresenta exemplos visuais.

Tabela 1: Detecções por imagem com os limiares padrão ($s_{\min} = 0,5$ e $\tau_{\text{IoU}} = 0,45$).

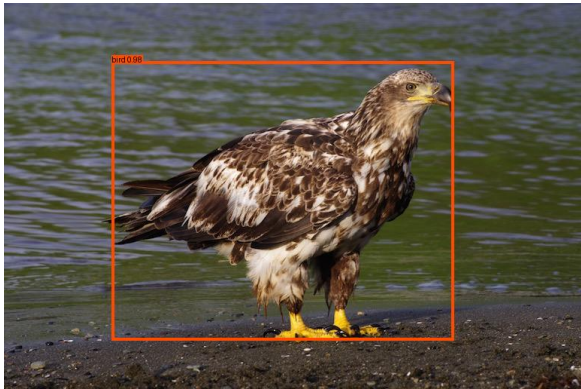
Imagem	Objetos	Classes detectadas
cat.jpg	1	cat
city_scene.jpg	15	person ($\times 6$), car ($\times 3$), traffic light ($\times 3$), truck ($\times 2$)
dog.jpg	3	bicycle, dog, truck
dog2.jpg	7	dog, person ($\times 4$), bicycle, umbrella
eagle.jpg	1	bird
food.jpg	18	bowl ($\times 10$), spoon ($\times 5$), diningtable, fork
giraffe.jpg	2	giraffe, zebra
horses.jpg	4	horse ($\times 4$)
motorbike.jpg	2	motorbike, person
person.jpg	3	person, dog, horse
surf.jpg	2	person, surfboard
wine.jpg	4	wine glass, bottle, chair ($\times 2$)



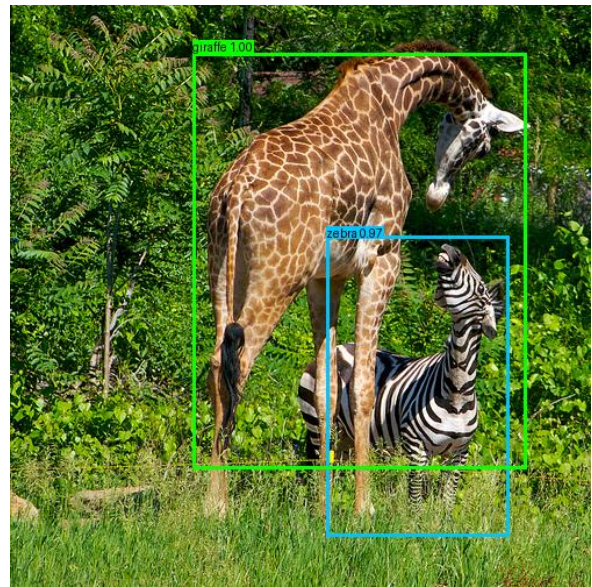
(a) dog.jpg: 3 objetos.



(b) horses.jpg: 4 cavalos.



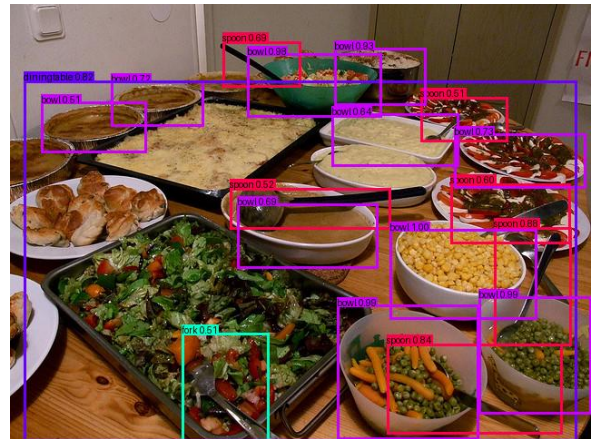
(c) eagle.jpg: 1 bird.



(d) giraffe.jpg: girafa e zebra.



(e) city_scene.jpg: 15 objetos.



(f) food.jpg: 18 objetos.

Figura 2: Exemplos de detecção com os limiares padrão.

4.1 Variação dos limiares

Para investigar o efeito dos limiares de pós-processamento (item 3 do enunciado), repetimos a inferência sobre `horses.jpg` variando o limiar de IoU enquanto mantínhamos $s_{\min} = 0,5$. A Tabela 2 mostra o número de caixas mantidas em cada cenário, e a Figura 3 apresenta a comparação visual.

Tabela 2: Efeito do limiar de IoU em `horses.jpg` ($s_{\min} = 0,5$).

τ_{IoU}	Caixas mantidas	Comentário
0,10	3	supressão excessiva, um cavalo é eliminado
0,30	3	ainda agressivo
0,45	4	valor padrão; resultado correto
0,60	4	resultado igual a 0,45
0,90	13	duplicatas em quase todos os cavalos

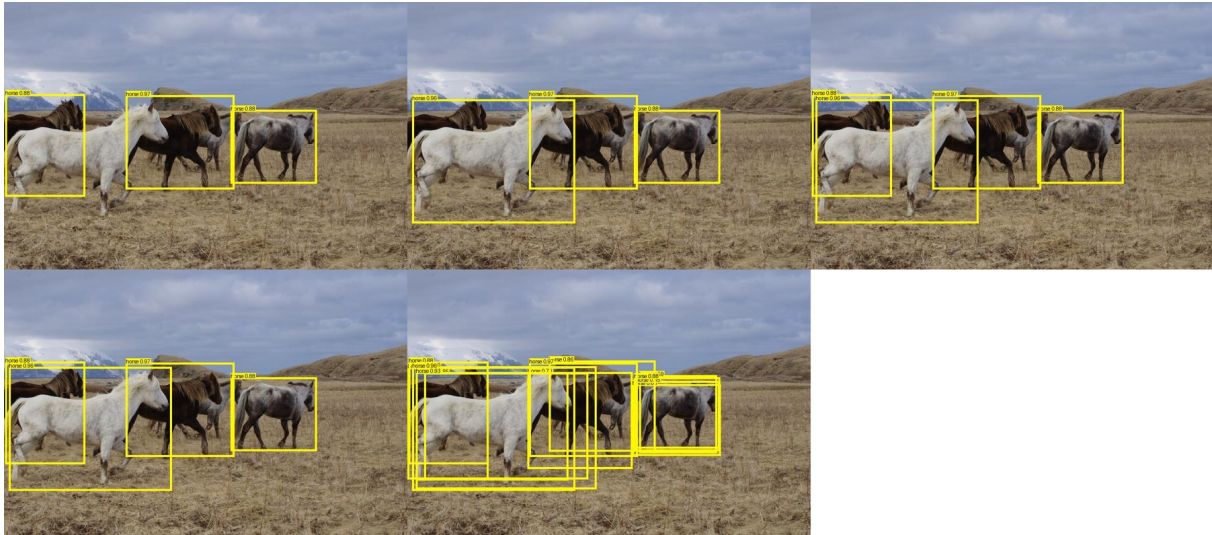


Figura 3: Variação do limiar de IoU sobre `horses.jpg`. Limiares baixos suprimem caixas verdadeiras de objetos próximos; limiares altos preservam duplicatas.

Em paralelo, fixamos $\tau_{\text{IoU}} = 0,45$ e variamos o limiar de *score* sobre a mesma imagem (Tabela 3). Como os quatro cavalos são detectados com confiança alta (todas acima de 0,88), o número de detecções permanece estável até $s_{\min} = 0,80$.

Tabela 3: Efeito do limiar de *score* em `horses.jpg` ($\tau_{\text{IoU}} = 0,45$).

$s_{\min}$	Caixas mantidas
0,20	4
0,40	4
0,60	4
0,80	4

5 Discussão

Os resultados confirmam que o YOLOv3 produz detecções de boa qualidade na maior parte das imagens, com alta confiança nos objetos principais. Em `dog.jpg`, por exemplo, as três classes esperadas (cachorro, bicicleta e caminhão) são identificadas com *scores* de 0,99, 1,00 e 0,92, respectivamente, em concordância com a figura de referência do enunciado. O mesmo acontece em `cat.jpg`, `eagle.jpg` e `horses.jpg`, em que as classes pertencem a categorias bem representadas no COCO.

Em cenas mais complexas, surgem comportamentos esperados e também algumas falhas. Em `city_scene.jpg`, o detector recupera 15 objetos, com pessoas, carros, semáforos e caminhões. Já em `giraffe.jpg`, observa-se uma confusão sistemática: o flanco listrado de uma das girafas é classificado como zebra com confiança 0,97. Esse tipo de erro, embora chamativo, é coerente com a forma como redes treinadas no COCO aprendem texturas e silhuetas, e ilustra o limite do que se consegue obter sem treinamento adicional sobre dados específicos do domínio. Em `dog2.jpg`, o detector encontra um guarda-chuva onde, na verdade, há um objeto similar mas distinto, possivelmente por viés de cena.

5.1 Papel dos limiares

A análise da Tabela 2 torna explícito o *trade-off* associado ao limiar de IoU. Quando τ é muito baixo (0,10), a NMS é agressiva: como dois cavalos lado a lado podem ter caixas com sobreposição não negligenciável, parte deles é descartada. Quando τ é muito alto (0,90), a NMS deixa de cumprir seu papel: as duplicatas geradas pelas três escalas e por âncoras vizinhas não são suprimidas e o número de caixas explode para 13. O valor $\tau = 0,45$, padrão na literatura, corresponde a um equilíbrio adequado para esta cena.

O limiar de *score*, por sua vez, atua antes da NMS, como um filtro grosseiro de confiança. Em cenas em que os objetos têm alta confiança (caso de `horses.jpg`), seu efeito é pequeno; em cenas com objetos pequenos ou atípicos, valores mais permissivos podem aumentar a cobertura, mas trazem mais falsos positivos. Em produção, ambos os limiares costumam ser ajustados em conjunto sobre uma base de validação, em função das exigências de precisão e revocação da aplicação.

5.2 Limitações e melhorias possíveis

A principal limitação do nosso pipeline é, por construção, o uso de pesos treinados sobre o COCO e nunca refinados. Isso restringe as classes detectáveis às 80 categorias originais e penaliza imagens cujas distribuições difiram daquela base. Algumas direções naturais de melhoria são:

- Migrar para versões mais recentes da família YOLO (v5, v7, v8), que incorporam técnicas como *mosaic augmentation*, âncoras adaptativas, perdas baseadas em CIoU e cabeças desacopladas [4, 5], e tendem a entregar precisão superior com latência semelhante.

- Substituir o NMS clássico pelo *Soft-NMS* [13], que reduz o *score* das caixas sobrepostas em vez de eliminá-las abruptamente, atenuando o problema observado em τ baixo.
- Realizar *fine-tuning* sobre dados específicos do domínio quando a aplicação exigir classes ausentes do COCO.
- Incluir uma etapa de validação automática (cálculo de *mAP* contra uma base de teste anotada) para medir, e não apenas inspecionar visualmente, o impacto de cada escolha.

Referências

- [1] Redmon, J., Divvala, S., Girshick, R., Farhadi, A. *You Only Look Once: Unified, Real-Time Object Detection*. CVPR, 2016.
- [2] Redmon, J., Farhadi, A. *YOLO9000: Better, Faster, Stronger*. CVPR, 2017.
- [3] Redmon, J., Farhadi, A. *YOLOv3: An Incremental Improvement*. arXiv:1804.02767, 2018.
- [4] Bochkovskiy, A., Wang, C.-Y., Liao, H.-Y. M. *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv:2004.10934, 2020.
- [5] Jocher, G. et al. *YOLOv5*. Repositório oficial: <https://github.com/ultralytics/yolov5>, 2020.
- [6] Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., Belongie, S. *Feature Pyramid Networks for Object Detection*. CVPR, 2017.
- [7] He, K., Zhang, X., Ren, S., Sun, J. *Deep Residual Learning for Image Recognition*. CVPR, 2016.
- [8] Lin, T.-Y. et al. *Microsoft COCO: Common Objects in Context*. ECCV, 2014.
- [9] Girshick, R., Donahue, J., Darrell, T., Malik, J. *Rich feature hierarchies for accurate object detection and semantic segmentation*. CVPR, 2014.
- [10] Ren, S., He, K., Girshick, R., Sun, J. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. NIPS, 2015.
- [11] Liu, W. et al. *SSD: Single Shot MultiBox Detector*. ECCV, 2016.
- [12] Carion, N. et al. *End-to-End Object Detection with Transformers*. ECCV, 2020.
- [13] Bodla, N., Singh, B., Chellappa, R., Davis, L. S. *Soft-NMS: Improving Object Detection With One Line of Code*. ICCV, 2017.
- [14] Zou, Z., Chen, K., Shi, Z., Guo, Y., Ye, J. *Object Detection in 20 Years: A Survey*. Proceedings of the IEEE, 111(3), 257-276, 2023.
- [15] Paszke, A. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS, 2019.